



UI & UX

INTERNACIONALIZAÇÃO, REGIONALIZAÇÃO E ACESSIBILIDADE

Prof. Me. Gustavo Ferreira Palma

PAUTA

- Internacionalização (i18n) e localização/regionalização (l10n)
- Resources, strings, plurais, datas, números e moedas
- Idiomas por aplicativo e layouts RTL
- Acessibilidade e tecnologias assistivas
- Semântica, `contentDescription`, foco e áreas de toque
- Testes práticos com Jetpack Compose

INTERNACIONALIZAÇÃO

UMA APLICAÇÃO NÃO VIVE SÓ NO NOSSO CONTEXTO

- Idioma é apenas uma parte da experiência.
- Datas, números, moedas, unidades e direção de leitura também variam.
- Uma interface pode estar traduzida e ainda assim parecer “estrangeira”.
- Projetar para outros contextos começa antes da tradução.

I18N E L10N

- **Internacionalização (i18n):** preparar o software para ser adaptável.
- **Localização (l10n):** adaptar conteúdo e apresentação a uma localidade.
- Regionalização envolve idioma, região, convenções e formatos.
- Uma boa i18n reduz alterações de código durante a localização.

O ANTI-PADRÃO: TEXTO NO CÓDIGO

```
Text("Configurações")  
Button(onClick = { salvar() }) {  
    Text("Salvar")  
}
```

Como traduzir sem alterar o código?

RESOURCES NO ANDROID

```
<!-- res/values/strings.xml -->
<resources>
    <string name="settings">Configurações</string>
    <string name="save">Salvar</string>
</resources>
```

```
Text(stringResource(R.string.settings))
```

RECURSOS ALTERNATIVOS

```
res/  
├── values/strings.xml  
├── values-en/strings.xml  
├── values-es/strings.xml  
└── values-pt-rBR/strings.xml
```

- O Android seleciona o recurso mais adequado à configuração.
- O conjunto padrão continua sendo essencial como fallback.

PORTUGUÊS E INGLÊS

values/strings.xml

```
<string name="result">Resultado</string>  
<string name="clear">Limpar</string>
```

values-en/strings.xml

```
<string name="result">Result</string>  
<string name="clear">Clear</string>
```

STRINGS COM PARÂMETROS

Não monte frases localizáveis por concatenação.

```
<string name="welcome">Olá, %1$s!</string>
```

```
Text(  
    stringResource(R.string.welcome, nome)  
)
```

A ordem das palavras pode mudar em outro idioma.

PLURALIZAÇÃO

```
<plurals name="items">
  <item quantity="one">%d item</item>
  <item quantity="other">%d itens</item>
</plurals>
```

```
val texto = pluralStringResource(
    R.plurals.items,
    quantidade,
    quantidade
)
```

POR QUE PLURAL NÃO É APENAS “ADICIONAR S”?

- Idiomas possuem regras diferentes de pluralização.
- O Android trabalha com categorias como one, few, many e other.
- A regra pertence à língua, não à lógica de negócio.

UNIDADES DE MEDIDA NO ANDROID

Diferentes elementos da interface utilizam
unidades de medida diferentes

Unidade	Significado	Uso
px	Pixel	Pixel físico da tela
dp	Density-independent Pixel	Dimensões, espaçamentos e componentes
sp	Scale-independent Pixel	Principalmente tamanho de textos

dp

considera a densidade da tela.

sp

considera a densidade

e a preferência de tamanho de fonte do usuário

DP × SP × PX



INTERFACE → DP

```
Modifier  
  .width(120.dp)  
  .height(48.dp)  
  .padding(16.dp)
```

Botões, margens, espaçamentos, ícones e dimensões da interface.



TEXTO → SP

```
Text(  
  text = "Calculadora",  
  fontSize = 24.sp  
)
```

O tamanho pode acompanhar as **preferências de acessibilidade** configuradas pelo usuário.

Evite definir a interface diretamente em px.

POR QUE TEXTOS USAM SP?

O usuário pode alterar o **tamanho das fontes** nas configurações de acessibilidade do Android.

Calculadora
Escala padrão → **Calculadora**
Fonte ampliada

Uma interface acessível deve continuar funcionando quando o **texto aumenta de tamanho**.

REGIONALIZAÇÃO

LOCALE

- Representa preferências linguísticas e regionais.
- pt - BR: português usado no Brasil.
- en - US: inglês usado nos Estados Unidos.
- en - GB: inglês usado no Reino Unido.
- Mesmo idioma não significa mesmas convenções.

NÚMEROS: O MESMO VALOR, OUTRA ESCRITA

Brasil: 1.234,56

Estados Unidos: 1,234.56

```
val formatter = NumberFormat.getNumberInstance()  
val texto = formatter.format(1234.56)
```

MOEDA

```
val formatter = NumberFormat.getCurrencyInstance()  
val preco = formatter.format(1234.50)
```

- Separadores e símbolo dependem da localidade.
- Não concatene "R\$ " + valor.

DATAS E HORÁRIOS

11/09/2026 — leitura comum no Brasil

09/11/2026 — pode representar a mesma data nos EUA

```
val formatador = DateFormat.getDateInstance(  
    DateFormat.MEDIUM  
)  
val texto = formatador.format(Date())
```

EVITE ASSUMIR FORMATOS

- Dia/mês/ano não é universal.
- Relógio de 12 h e 24 h varia conforme contexto.
- Vírgula nem sempre é separador decimal.
- Unidades também podem precisar de adaptação.

IDIOMA POR APLICATIVO

- Android moderno permite escolher um idioma específico para cada app.
- No Android 13+, essa preferência integra-se às configurações do sistema.
- É possível declarar os idiomas suportados e oferecer um seletor no app.
- Isso é diferente de simplesmente trocar textos manualmente.

DIREÇÃO DE LEITURA: RTL

LTR

RTL

← início fim → ← fim início →

- Árabe e hebraico usam direção da direita para a esquerda.
- Prefira `start/end a left/right` em layouts.

INTERNACIONALIZAR É MAIS QUE TRADUZIR

- Textos podem crescer ou diminuir.
- Botões precisam comportar traduções maiores.
- Imagens com texto embutido dificultam localização.
- Layouts rígidos quebram mais facilmente.

ACESSIBILIDADE

ACESSIBILIDADE É PARTE DE UX

- Usuários podem interagir de maneiras diferentes da nossa.
- Deficiências podem ser permanentes, temporárias ou situacionais.
- Visão, audição, mobilidade e cognição influenciam a interação.
- A interface deve comunicar significado além da aparência.

TECNOLOGIAS ASSISTIVAS

- **TalkBack**: leitor de tela do Android.
- Acesso com interruptor e outras formas alternativas de entrada.
- Ampliação, tamanho de fonte e alto contraste.
- Teclado físico e navegação por foco também importam.

O QUE O TALKBACK “ENXERGA”?

Não a tela como nós vemos, mas uma **árvore semântica**.

- Texto
- Descrição
- Papel do componente
- Estado
- Ações disponíveis

SEMÂNTICA NO COMPOSE

```
Modifier.semantics {  
  contentDescription = "Adicionar produto"  
}
```

- Semântica fornece significado aos serviços de acessibilidade e testes.
- Componentes Material já carregam muita semântica automaticamente.

CONTENTDESCRIPTION

Um ícone visualmente óbvio pode ser invisível para um leitor de tela.

```
Icon(  
    imageVector = Icons.Default.Delete,  
    contentDescription = stringResource(  
        R.string.cd_delete  
    )  
)
```

DESCRIÇÃO BOA × DESCRIÇÃO RUIM

✗ contentDescription = "Ícone de lixeira vermelho"

✓ contentDescription = "Excluir arquivo"

- Descreva significado/ação, não detalhes visuais desnecessários.

QUANDO USAR NULL?

```
Icon(  
  imageVector = Icons.Default.Star,  
  contentDescription = null  
)
```

- Elementos puramente decorativos não precisam ser anunciados.
- Evite informação duplicada quando o texto vizinho já comunica tudo.

UM BOTÃO NÃO PRECISA FALAR DUAS VEZES

```
Button(onClick = onSave) {  
    Icon(  
        Icons.Default.Save,  
        contentDescription = null  
    )  
    Text(stringResource(R.string.save))  
}
```

O texto do botão já fornece o significado da ação.

DESCRIÇÃO TAMBÉM DEVE SER LOCALIZADA

Não faça:

```
contentDescription = "Excluir"
```

Faça:

```
contentDescription = stringResource(  
    R.string.cd_delete  
)
```

Internacionalização e acessibilidade se encontram aqui.

ESTADOS TAMBÉM PRECISAM SER COMUNICADOS

```
Modifier.semantics {  
  stateDescription = if (ligado)  
    "Ativado"  
  else  
    "Desativado"  
}
```

Quando necessário, exponha ao serviço assistivo o estado que o usuário precisa conhecer.

AGRUPANDO INFORMAÇÃO

```
Row(  
  modifier = Modifier.semantics(  
    mergeDescendants = true  
  ) { }  
) {  
  Icon(...)  
  Text("Wi-Fi")  
  Text("Conectado")  
}
```

Elementos relacionados podem formar uma única unidade semântica.

ÁREA DE TOQUE

24 dp pode ser um bom tamanho visual para um ícone.

48 × 48 dp é a referência mínima para a área interativa.

O alvo de toque pode ser maior que o desenho do ícone.

USE COMPONENTES QUE JÁ AJUDAM

```
IconButton(  
    onClick = onDelete  
) {  
    Icon(  
        Icons.Default.Delete,  
        contentDescription = stringResource(  
            R.string.cd_delete  
        )  
    )  
}
```

Material Components já incorporam várias decisões de acessibilidade.

COR NÃO PODE SER A ÚNICA MENSAGEM

 Campo vermelho

  Campo vermelho + “Valor inválido”

- Daltonismo é um exemplo, mas a regra beneficia muitos contextos.
- Use texto, forma, ícone ou outra pista adicional.

CONTRASTE

- Texto pequeno: referência de contraste de **4,5:1**.
- Texto grande e gráficos essenciais: **3:1**.
- Não assuma que “parece legível para mim” é um teste suficiente.
- Temas claro, escuro e customizados precisam ser avaliados.

ESCALA DE FONTE

- O usuário pode aumentar o tamanho do texto no sistema.
- Layouts com alturas fixas podem cortar conteúdo.
- Prefira componentes flexíveis e papéis de `MaterialTheme.typography`.
- Teste o aplicativo com fontes grandes.

FOCO E NAVEGAÇÃO

- Existe uma ordem lógica para percorrer a interface?
- O foco é visível?
- É possível chegar às ações importantes sem depender apenas do toque?
- Componentes customizados exigem atenção especial.

TEXTO E LINGUAGEM TAMBÉM SÃO ACESSIBILIDADE

- Rótulos claros reduzem carga cognitiva.
- Mensagens de erro devem explicar o problema e, quando possível, a solução.
- Evite jargão quando ele não agrega significado.
- Consistência de termos ajuda todos os usuários.

APLICANDO NO ANDROID

EXEMPLO: BOTÃO ACESSÍVEL E LOCALIZÁVEL

```
IconButton(onClick = onClear) {  
    Icon(  
        imageVector = Icons.Default.Clear,  
        contentDescription = stringResource(  
            R.string.cd_clear_calculator  
        )  
    )  
}
```

```
<string name="cd_clear_calculator">  
    Limpar calculadora  
</string>
```

EXEMPLO: VALOR DO ACELERÔMETRO

```
Text(  
    text = stringResource(  
        R.string.axis_value,  
        "X",  
        valorX  
    ),  
    style = MaterialTheme.typography.titleLarge  
)
```

```
<string name="axis_value">%1$s: %2$.2f m/s2</string>
```

EXEMPLO: INFORMAÇÃO SEM DEPENDER DA COR

```
if (erro) {  
    Row {  
        Icon(  
            Icons.Default.Warning,  
            contentDescription = null  
        )  
        Text(stringResource(R.string.invalid_value))  
    }  
}
```

O ícone é decorativo porque o texto já anuncia a informação.

CHECKLIST RÁPIDO

- Há textos fixos no Kotlin?
- Datas, números e moedas respeitam Locale?
- Ícones de ação possuem descrição?
- Elementos decorativos estão fora da leitura desnecessária?
- A informação depende somente de cor?
- Alvos de toque são adequados?
- Fonte grande quebra o layout?

TESTE COM TALKBACK

1. Ative o TalkBack no dispositivo.
2. Percorra a tela sem olhar para ela.
3. Observe ordem, descrições, estados e ações anunciadas.
4. Corrija a semântica e teste novamente.

O teste manual continua sendo indispensável.

TESTE DE INTERNACIONALIZAÇÃO

1. Execute o app em português.
2. Troque para inglês.
3. Observe textos maiores e menores.
4. Teste datas, números e moedas.
5. Procure textos que ficaram no idioma original.

DESAFIO

Use o aplicativo por alguns minutos com **TalkBack** ativado e **sem olhar para a tela**.

Quais decisões visuais deixaram de existir? Quais informações semânticas passaram a ser essenciais?

OBRIGADO!

Podem me encontrar aqui:

- gustavo.palma@unisal.edu.br
- www.gustavopalma.com.br